LEARNING GUIDE · WRITTEN FROM THE ACTUAL CODEBASE

React, for someone who already thinks in Angular

A guided tour of the Infraflow frontend (`infraflow` — Vite + React 18 + TypeScript) that teaches React from first principles by reading real components, real state management, and real routing that already exist in this repository — with every idea anchored to its closest Angular equivalent.

`react 18` `typescript` `vite` `zustand` `reactflow` `no react-router`

Prepared for: umailkuldeep@gmail.com

Source: this repository's `src/` tree, current as of 2026-08-25

Prior knowledge assumed: Angular (components, services, RxJS, DI, templates)

Contents

1. The mental model shift — Angular → React, in one table
2. How this app is put together — File anatomy, Vite, the entry point
3. JSX — a template that is just JavaScript — vs. Angular's template DSL
4. Components & props — `Sidebar.tsx` vs. `@Input()`
5. State with `useState` — `AuthPage.tsx` vs. component fields + change detection
6. Side effects with `useEffect` — `App.tsx` vs. lifecycle hooks & RxJS
7. Derived values: `useMemo` — `Sidebar.tsx`, `Canvas.tsx` vs. pure pipes
8. Global state without NgRx: Zustand — `diagramStore.ts` vs. services + `BehaviorSubject`
9. Routing without a router — `App.tsx` vs. `RouterModule`
10. Lists and conditionals — `.map()` / `&&` vs. `*ngFor` / `*ngIf`
11. Forms: controlled inputs — `AuthPage.tsx` vs. `[(ngModel)]`
12. Context API — `canvasGraphContext.ts` vs. DI tokens
13. Refs & imperative escape hatches — `useRef` vs. `@ViewChild`
14. Code-splitting — `React.lazy` vs. lazy-loaded modules
15. Wrapping third-party libraries — ReactFlow, Monaco, react-three-fiber
16. TypeScript patterns you'll see everywhere — `types.ts` and friends
17. Styling: no view encapsulation — global CSS + theme classes
18. A guided reading path — open these files in this order
19. Exercises inside this exact repo — learn by changing real code
20. Cheat sheet — one page, pin it to your monitor

01 The mental model shift

Angular gives you a framework: a compiler, dependency injection, a router, forms modules, RxJS baked into the HTTP client. React gives you almost none of that — it is *just the view layer*. Everything you're used to Angular providing out of the box, this codebase either hand-rolls (routing in `App.tsx`) or pulls in as an ordinary npm package (`zustand` for state, `reactflow` for the diagram canvas). That's the single biggest adjustment: React is a library you compose, not a framework that dictates structure.

ANGULAR CONCEPT	REACT / THIS CODEBASE'S EQUIVALENT
Component class + separate HTML template	One function that returns JSX (markup and logic live together) — see Ch. 4
`@Input()` / `@Output()`	Function parameters (`props`), including callback props like `onToggleCollapsed` — Ch. 3
Component field + Angular's automatic change detection	`useState` — you call a setter, React re-renders — Ch. 5
`ngOnInit` / `ngOnChanges` / `ngOnDestroy`	`useEffect` with a dependency array and an optional cleanup function — Ch. 6
Pure pipe / computed signal	`useMemo` — Ch. 7
Injectable service + `BehaviorSubject`, or NgRx store	`zustand` store (`diagramStore.ts`) — Ch. 8
`RouterModule`, `routerLink`, route guards	Hand-written `if` chain on `window.location.pathname` in `App.tsx` — Ch. 9
`*ngFor` / `*ngIf`	`array.map()` / `{'{'}condition && …{'}'}` — Ch. 10
Reactive Forms / `[(ngModel)]`	"Controlled" inputs: `value` + `onChange` tied to state — Ch. 11
Dependency injection token / provider	`createContext` + `useContext` — Ch. 12
`@ViewChild`	`useRef` — Ch. 13
Lazy-loaded route module	`React.lazy()` + `<Suspense>` — Ch. 14
NgModule (declarations / imports / providers)	Nothing — you just `import` the function you need. No module system beyond ES modules.

Zone.js triggers change detection automatically	Nothing runs automatically. State only changes when *you* call a setter. This is the root cause of most "why didn't it re-render" confusion coming from Angular.

THE ONE IDEA TO INTERNALIZE FIRST

A React component is a plain function. React calls that function every time it thinks something might have changed, and the function's return value (JSX) describes what the UI should look like *right now*. There is no "update the DOM to match" step you write yourself, and there is no persistent instance holding mutable fields between renders the way an Angular component class does — instead, `useState / useRef` are how a function "remembers" things across those repeated calls.

02 How this app is put together

No Angular CLI, no `angular.json`. The build tool is Vite, configured through `package.json` scripts — `npm run dev` starts a dev server, `npm run build` type-checks with `tsc` and then bundles with Vite.

```
src/ main.tsx ← the one line that mounts React (like main.ts + bootstrapApplication) App.tsx ← top-level component; also does hand-rolled routing styles.css, theme.ts ← global styles + theme-mode logic (no Angular Material here) store/diagramStore.ts ← the "app-wide service" — a Zustand store components/ ← shared, reusable UI: Canvas, Sidebar, Toolbar, modals… dashboard/ ← the /dashboard feature area, its own pages/, apis, components landing/, auth/, legal/, settings/ ← other top-level "routes" utils/ ← plain functions: validation, layout math, TS types, api clients data/awsServices.ts ← static catalogue data (AWS service definitions)
```

◆ src/main.tsx

```tsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <ErrorBoundary>
      <ReactFlowProvider>
        <App />
      </ReactFlowProvider>
    </ErrorBoundary>
  </React.StrictMode>,
);
```

ANGULAR

`bootstrapApplication(AppComponent, config)` in `main.ts`, wired to `index.html`'s `<app-root>`.

REACT

`createRoot(el).render(<App />)` in `main.tsx`, wired to `index.html`'s `<div id="root">`. Everything nested inside is a "provider" — same idea as Angular providers, just as JSX-wrapped components instead of a DI array.

`<React.StrictMode>` is a dev-only helper that intentionally double-invokes components/effects to surface bugs (like an Angular dev-mode check that flags a value changing after it was already read). `ErrorBoundary` is a component that catches render errors in its children — the closest thing React has to a global error handler for the component tree, but you write it yourself (see `src/components/ErrorBoundary.tsx`) rather than it being framework-provided. `ReactFlowProvider` is a context provider from the `reactflow` library, exactly the shape you'll build yourself in Chapter 12.

03 JSX — a template that is just JavaScript

Angular splits a component into a class (logic) and a separate template file (markup), connected by string-based binding syntax the Angular compiler understands. React components inline the markup as JSX — syntax that looks like HTML but compiles to plain `React.createElement()` calls. Because it's real JavaScript, there's no separate template language to learn: `if`, `map`, ternaries, and template literals all just work inside `{'{'} … {'}'}`.

ANGULAR TEMPLATE

```
<div *ngIf="isCollapsed" class="sidebar
sidebar--collapsed">
  <button (click)="onToggleCollapsed.emit()">
    <panel-left-open-icon></panel-left-open-
icon>
  </button>
</div>
```

THIS REPO — SIDEBAR.TSX

```
if (isCollapsed) {
  return (
    <aside className="sidebar sidebar--
collapsed">
      <button onClick={onToggleCollapsed}>
        <PanelLeftOpen size={18} />
      </button>
    </aside>
  );
}
```

Three small but constant differences you'll hit immediately, all visible in `Sidebar.tsx`:

- **`class`** → **`className`**, because `class` is a reserved JavaScript keyword.
- **Event bindings are camelCase and take a function reference**, not a string expression: `onClick={'{'}onToggleCollapsed{'}'}`, not `(click)="onToggleCollapsed()"`.
- **Icons are components, not directives** — `<PanelLeftOpen size={'{'}18{'}'} />` from the `lucide-react` package is a function that returns an `<svg>`, imported like any other component. There is no icon registry/module to declare.

> **READING HABIT TO BUILD**
>
> Whenever you see curly braces inside JSX — `{'{'}query{'}'}`, `{'{'}service.name{'}'}`, `{'{'}isAllowed ? '' : 'service-item--locked'{'}'}` — that's a JavaScript expression being interpolated, exactly the role Angular's `{'{{ }}'}` interpolation and property bindings (`[prop]="expr"`) play, just unified into one syntax that works both for text content and for attribute values.

04 Components & props

A React component is a function whose single argument is an object of "props" — the direct analogue of everything an Angular component receives via `@Input()`, plus its `@Output()` event emitters, collapsed into one plain object with no decorators.

◆ src/components/Sidebar.tsx — full component signature

```
function Sidebar({ isCollapsed = false, onToggleCollapsed, user }: {
  isCollapsed?: boolean;
  onToggleCollapsed?: () => void;
  user?: AuthUser | null;
}) {
  // ...component body...
}

export default Sidebar;
```

Compare the pieces directly:

ANGULAR	REACT (THIS FILE)
`@Input() isCollapsed = false;`	`isCollapsed = false` destructured from the props object, with a default value using plain JS default-parameter syntax
`@Output() toggleCollapsed = new EventEmitter<void>();`	`onToggleCollapsed?: () => void` — just a function prop. The parent passes a callback; there's no `EventEmitter` class, no `.emit()`.
The type is declared once, on the class, via decorators	The type is a plain TypeScript object type written inline (or extracted to a named `type Props = {'{'} … {'}'}` in bigger components)

Using it from a parent is just calling it like an HTML tag and passing attributes:

```
<Sidebar
  isCollapsed={sidebarCollapsed}
  onToggleCollapsed={() => setSidebarCollapsed(!sidebarCollapsed)}
  user={currentUser}
/>
```

> **WHERE "THE TEMPLATE" WENT**
>
> There's no separate `sidebar.component.html` — the JSX returned at the bottom of the function *is* the template, and it's written directly against the component's local variables and props with no binding-syntax boundary to cross. This is why the file is `.tsx` (TypeScript + JSX) rather than `.ts`.

05 State with `useState`

In Angular, a component field is just a class property — you assign to it, and Angular's zone-based change detection notices and re-renders. React has no such automatic detection: a plain variable assignment does **nothing** visible. State has to be created with `useState`, and changed only by calling the setter function it returns — that call is what schedules a re-render.

◆ src/auth/AuthPage.tsx — a form with many independent state values

```
function AuthPage({ mode, theme, onToggleTheme }) {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [error, setError] = useState('');
  // ...12 useState calls total in this one component...
}
```

`useState('')` returns a two-element array: the *current* value, and a setter function that replaces it. `const [x, setX] = useState(initial)` is array-destructuring — you'll see this exact shape hundreds of times across the codebase, always named `[thing, setThing]` by convention.

ANGULAR	REACT
`email = '';` then `this.email = value;` anywhere, and the template just re-reads `email` next digest cycle	`const [email, setEmail] = useState('')` ; you must call `setEmail(value)` — direct mutation like `email = value` is silently ignored by React
One component instance persists; its fields live for the component's lifetime	The *function* re-runs on every render; `useState` is what makes a value survive between those calls instead of resetting to its initial value each time
Many fields on one class, all mutable together	Many separate `useState` calls, each independently triggering a re-render when its own setter is called — this is why AuthPage has a dozen of them instead of one big object

The controlled-input wiring you'll see everywhere reads directly from and writes directly to this state (full pattern in Chapter 11):

```
<input value={email} onChange={(event) => setEmail(event.target.value)} />
```

> **COMMON TRIP-UP COMING FROM ANGULAR**
>
> Calling a setter does not update the value synchronously in the same function call. `setEmail('x'); console.log(email)` still logs the *old* value — React schedules the re-render for after the current function finishes. The new value only exists once the component function runs again (i.e., on the next render), the same

> way an Angular value only reflects a change after the next change-detection pass — except here you can see exactly which pass, because you triggered it explicitly.

06 Side effects with `useEffect`

`useEffect` is React's single hook for everything Angular splits across `ngOnInit`, `ngOnChanges`, and `ngOnDestroy`: run some code after render, optionally re-run it when specific values change, and optionally clean up before the next run or before the component unmounts.

◆ src/App.tsx — auth guard on a protected route

```
useEffect(() => {
  if (!requiresAuth) return;
  let isCurrent = true;

  async function validateProtectedAccess() {
    setIsValidatingProtectedRoute(true);
    const user = await validateStoredSession();
    if (!isCurrent) return;         // component moved on — ignore this stale response
    if (!user) { window.location.replace(`/login?next=...`); return; }
    setIsProtectedRouteAllowed(true);
  }

  void validateProtectedAccess();

  return () => { isCurrent = false; };   // cleanup — the ngOnDestroy-equivalent part
}, [path, requiresAuth]);                    // dependency array — the ngOnChanges-equivalent part
```

PIECE	ANGULAR EQUIVALENT
The function body runs after every render where a dependency changed	`ngOnChanges()` — but you don't compare `SimpleChanges` yourself; React does the "did a dependency change" check for you
`[path, requiresAuth]` dependency array	No direct equivalent — closest is manually deciding what to react to inside `ngOnChanges`; here it's declarative: "only rerun when either of these values differs from last time"
Empty array `[]` (not used here, but common elsewhere)	Runs once, right after first render — the `ngOnInit` case
No array at all	Runs after *every* render — rare, usually a bug when unintended
The returned function `() => { isCurrent = false }`	`ngOnDestroy()` — but it also runs before every re-run of the effect itself, not just on unmount, which is why `isCurrent` is needed to ignore a stale async response from a previous run (a race Angular developers usually solve with `switchMap` / `takeUntil` in RxJS instead)

> **WHY `ISCURRENT` INSTEAD OF RXJS**

> Angular developers instinctively reach for `switchMap` to cancel a stale in-flight request. React has no built-in cancellation primitive for a plain `async / await` function, so the idiom you'll see repeatedly in this codebase is a boolean flag captured by the effect's closure, flipped to `false` in the cleanup, and checked after every `await`. It's more manual than RxJS, but it's the same problem (ignore responses for requests you no longer care about) solved without pulling in a stream library for it.

07 Derived values: `useMemo`

Angular templates re-evaluate an expression like `services | filter:query` on every change detection cycle unless you write a pure pipe or use a signal-based computed value. React has the same "don't recompute this expensive thing unless its inputs changed" need, solved with `useMemo`.

◆ src/components/Sidebar.tsx

```
const filtered = useMemo(() => {
  const normalized = query.trim().toLowerCase();
  return awsServices.filter((service) =>
    !normalized || `${'$'}{service.name} ${'$'}{service.category} ${'$'}
{service.shortName}`.toLowerCase().includes(normalized)
  );
}, [query]);   // only re-filter when `query` itself changes
```

`useMemo(computeFn, deps)` caches the result of `computeFn` and only recomputes it when a value in `deps` has changed since the last render — otherwise it returns the previously cached result, skipping the work entirely. Functionally this is a pure pipe's job (Angular only re-runs a pure pipe when its input reference changes), just declared at the point of use instead of as a separately registered class.

◆ src/components/Canvas.tsx — memoizing a derived UI band from a raw zoom number

```
const { zoom } = useViewport();
// Re-render node cards only when the LOD *band* changes, not on every fractional zoom tick
const lodBand = useMemo(() => lodBandForZoom(zoom), [zoom]);
```

> **THE PERFORMANCE REASONING IS WORTH READING IN FULL**
>
> Look at the comment above `lodBand` in the real file (`src/components/Canvas.tsx:76-78`): the point isn't just "cache this computation," it's that memoizing collapses a continuously-changing number (zoom) into one of three stable string values, so anything consuming `lodBand` via context only re-renders when the *band* changes, not on every pixel of zooming. This combination of `useMemo` + Context is a recurring optimization pattern in this codebase — see Chapter 12.

There's a sibling hook, `useCallback(fn, deps)`, which does the same caching but for a *function reference* instead of a value — used so a callback passed as a prop doesn't change identity (and therefore doesn't cause a child to needlessly re-render) unless something it depends on actually changed.

08 Global state without NgRx: Zustand

This app's entire diagram-editing state — nodes, edges, selection, undo history, validation results — lives in one `zustand` store: `src/store/diagramStore.ts`. Zustand plays the role an injectable Angular service with a `BehaviorSubject` (or an NgRx store) would play: one place holding shared state, read and mutated from anywhere in the component tree without prop-drilling.

◆ src/store/diagramStore.ts — shape (trimmed)

```
type DiagramStore = {
  nodes: AwsNode[];
  edges: AwsEdge[];
  selectedNodeId?: string;
  isDirty: boolean;
  isValidated: boolean;
  history: DiagramSnapshot[];
  future: DiagramSnapshot[];
  // ...and the "methods" — actions that mutate the above:
  setSelection: (nodeId?: string, edgeId?: string) => void;
  addServiceNode: (serviceId: string, position: XYPosition) => void;
  onNodesChange: (changes: NodeChange[]) => void;
  undo: () => void;
  redo: () => void;
  validate: () => ValidationIssue[];
  // ...30+ actions total, all living beside the state they mutate
};

export const useDiagramStore = create<DiagramStore>((set, get) => ({ /* initial state + actions */
}));
```

Any component reads from it with a **selector** — a function picking just the slice it needs:

◆ src/components/Sidebar.tsx

```
const nodes = useDiagramStore((state) => state.nodes);
```

ANGULAR / NGRX	ZUSTAND (THIS REPO)
`@Injectable({'{'} providedIn: 'root' {'}'})` service, injected via constructor	`create<DiagramStore>(...)` exported once as `useDiagramStore`, imported directly wherever it's needed — no DI container, no constructor injection
`this.diagramService.nodes$.subscribe(...)`, then manual unsubscribe in `ngOnDestroy`	`useDiagramStore((state) => state.nodes)` — subscription and cleanup are both handled internally by the hook
`store.select(selectNodes)` (NgRx) to avoid over-subscribing to the whole state tree	The selector function `(state) => state.nodes` plays exactly this role — see the note below on why *which* selector you write matters a lot here

Dispatching an action / reducer	Calling an action method directly: `useDiagramStore.getState().undo()`, or destructuring it: `const undo = useDiagramStore((s) => s.undo)`
Immutable updates via reducers	Also done manually inside each action (using spreads / new arrays) — Zustand itself does not enforce immutability, the store's author does

> **SELECTOR GRANULARITY IS A REAL PERFORMANCE LEVER, NOT A STYLE PREFERENCE**
>
> `src/components/Canvas.tsx:88-90` has a comment worth reading verbatim: an earlier version called `useDiagramStore()` with no selector, which subscribed the component to *every* field in the store — so Canvas re-rendered (and recomputed the entire visible graph) on any state change at all, not just the ones it actually used. The fix was switching to individual selectors per field. There's no Angular parallel that bites this hard by default, because Angular's change detection doesn't have an equivalent "subscribed to literally everything" failure mode — this is a React-specific trap worth remembering the first time a component feels sluggish.

09 Routing without a router

There is no `react-router` in this app's dependencies — check `package.json` and you won't find it. Instead, `src/App.tsx` reads `window.location.pathname` once per render and builds up which page to show through a plain sequence of `if` statements.

◆ src/App.tsx — trimmed

```
function App() {
  const path = window.location.pathname;

  let page = <LandingPage theme={theme} onToggleTheme={toggleTheme} />;   // default route

  if (path === '/dashboard') {
    page = isValidatingProtectedRoute || !isProtectedRouteAllowed
      ? <main>Checking dashboard access...</main>
      : <DashboardShell theme={theme} onToggleTheme={toggleTheme} />;
  }
  if (path === '/references') page = <ReferenceDocsPage ... />;
  if (path === '/blog' || path === '/timeline') page = <TimelinePage ... />;
  if (path === '/login') page = <AuthPage mode="login" ... />;
  // ...more routes, then finally...

  return <div className={themeClassName}>{page}</div>;
}
```

ANGULAR	THIS APP
`const routes: Routes = [{'{'} path: 'dashboard', component: DashboardShell, canActivate: [authGuard] {'}'}]`	An `if (path === '/dashboard')` branch, with the "guard" being an inline `useEffect` that calls `validateStoredSession()` and redirects with `window.location.replace(...)` if it fails
`routerLink="/dashboard"`	A plain `<a href="/dashboard">` — full page navigation, not client-side routing
Route guards (`canActivate`) as injectable classes/functions	`requiresAuth` boolean + a `useEffect` per protected path, shown inline in `App.tsx` (see Chapter 6's walkthrough of this exact effect)
`ActivatedRoute` for params/query string	Read directly: `new URLSearchParams(window.location.search).get('resetToken')` , as seen in `AuthPage.tsx`

> **THIS IS A DELIBERATE, UNUSUAL CHOICE — DON'T ASSUME IT'S "HOW REACT APPS NORMALLY ROUTE"**
>
> Most real-world React apps use `react-router` (or a framework like Next.js) for client-side routing with no full page reloads. This codebase instead treats each top-level path as effectively its own mini single-page app, read once at mount from `window.location.pathname` — navigating between `/dashboard` and

> `/settings` does a real browser navigation, not an in-app route swap. Learn the pattern here because it's what's in front of you, but know that `react-router`'s `<Routes>` / `<Route>` / `useNavigate` is the far more common thing you'll meet in other React codebases and tutorials.

10 Lists and conditionals

No structural directives — `*ngFor` and `*ngIf` are replaced by ordinary JavaScript: `Array.prototype.map()` for lists, and `&&` / ternaries / early `return`s for conditionals.

◆ src/components/Sidebar.tsx — nested lists (categories → services)

```
{categories.map((category) => {
  const services = filtered.filter((s) => s.category === category);
  if (!services.length) return null;         // "render nothing" — the *ngIf equivalent
  return (
    <section key={category} className="service-group">
      <div className="service-group__title">{category}</div>
      {services.map((service) => (
        <div key={service.id} className={`service-item ${'$'}{isAllowed ? '' : 'service-item--
locked'}`}>
          <span className="truncate">{service.name}</span>
          {!isAllowed && <span className="service-item__lock">Locked</span>}
        </div>
      ))}
    </section>
  );
})}
```

ANGULAR	REACT
`*ngFor="let service of services; trackBy: trackById"`	`services.map((service) => <div key={'{'}service.id{'}'}>...</div>)` — `key` is mandatory and plays exactly the role of `trackBy`: it's how React matches array items across re-renders instead of re-creating every DOM node
`*ngIf="isAllowed"`	`{'{'}isAllowed && <span>...</span>{'}'}` — if `isAllowed` is falsy, the expression evaluates to `false`, and React renders nothing for a `false` / `null` / `undefined` child
`*ngIf="a; else b"`	Ternary: `{'{'}isAllowed ? <A/> : <B/>{'}'}`, or an early `return` from the component/render branch, as `Sidebar` does for its whole collapsed state
`[ngClass]="{'{'} 'active': isActive {'}'}"`	Plain string building: `className={'{'}`item ${'$'}{'{'}isActive ? 'active' : ''{'}'}`{'}'}` — there's no built-in class-binding helper, just template literals (this codebase does not use a classnames-style utility library)

FORGETTING `KEY` IS THE #1 BEGINNER MISTAKE

React will still render a list without `key`, but with a console warning, and — worse — it can silently reuse the wrong DOM node/state across re-renders when the list reorders (e.g., typing in an input inside item #2 can end up visually landing on item #3 after a resort). Always use a stable, unique id (`service.id`, `category`), never the array index, unless the list is truly static and never reorders.

11 Forms: controlled inputs

Angular's `[(ngModel)]` is two-way binding sugar for "set the input's value from the model, and write user keystrokes back into the model." React has no built-in two-way binding syntax — you wire both directions yourself, explicitly, which is why it's called a **controlled** input: the input's displayed value is fully controlled by React state, not by the DOM.

◆ src/auth/AuthPage.tsx

```
const [email, setEmail] = useState('');

<input
  type="email"
  value={email}
  onChange={(event) => setEmail(event.target.value)}
/>
```

ANGULAR

`<input [(ngModel)]="email">`

One directive does both the read and the write.

REACT

`value={'{'}email{'}'}` reads, `onChange={'{'}(e) => setEmail(e.target.value){'}'}` writes.
Two explicit halves, always paired together.

Submission follows the same "you write the plumbing" philosophy — `AuthPage.tsx`'s `handleSubmit` calls `event.preventDefault()` itself (there's no Angular-style `(ngSubmit)` that does it for you), validates by calling a plain function (`validateAuthForm`), and stores errors in a `useState` object rather than through Angular's `FormGroup` / `Validators` machinery:

```
async function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
  event.preventDefault();
  const validationErrors = validateAuthForm({ email, hasAcceptedLegal, isRegister, name, password
});
  setFieldErrors(validationErrors);
  if (Object.keys(validationErrors).length > 0) return;
  // ...call login()/register() from authClient.ts, an ordinary fetch wrapper...
}
```

NO REACTIVE FORMS MODULE HERE

This codebase does hand-written form state rather than a form library. That's a codebase choice, not a React limitation — libraries like `react-hook-form` exist and are common elsewhere, playing a role closer to Angular's Reactive Forms. Learn the manual pattern here first since it makes the underlying mechanics (what "controlled" actually means) impossible to skip past.

12 Context API

When a value needs to reach many components nested arbitrarily deep, passing it down as a prop through every intermediate layer ("prop drilling") gets unwieldy — the same problem Angular's DI hierarchy is designed to avoid. React's answer is `createContext` + a `<Provider>` + `useContext`.

◆ src/components/canvasGraphContext.ts — full file, three contexts

```
export const LodBandContext = createContext<LodBand>('full');

export function useLodBand(): LodBand {
  return useContext(LodBandContext);
}
```

Canvas provides a value further up the tree; any descendant node card calls `useLodBand()` to read it, no matter how deeply nested, with nothing threaded through the props of components in between:

PIECE	ANGULAR EQUIVALENT
`createContext(defaultValue)`	An `InjectionToken`, or an abstract/interface-typed service
`<LodBandContext.Provider value={lodBand}>`	A `providers: [{'{'} provide: TOKEN, useValue: ... {'}'}]` entry, scoped to a component subtree
`useContext(LodBandContext)` inside a descendant	Constructor injection: `constructor(@Inject(TOKEN) private band: LodBand)`

> **WHY THIS FILE HAS THREE SEPARATE CONTEXTS INSTEAD OF ONE**
>
> Read the comment at the top of the real file — it explains that `RelationshipBadgeContext`, `EdgeGeometryContext`, and `LodBandContext` are split deliberately because they want *opposite* re-render behavior: badge counts should trigger a re-render when they change, while obstacle rects change on every drag pointer tick but are exposed through a getter function whose identity never changes, so subscribing to that context costs nothing. This is a genuinely React-specific concern — in Angular, a service's fields don't carry this same "does reading this force an update" distinction, because change detection isn't driven by context value identity.

13 Refs & imperative escape hatches

`useRef` is React's `@ViewChild`: a way to reach a real DOM element (or hold a mutable value that should *not* trigger a re-render when it changes, unlike `useState`).

◆ src/components/Canvas.tsx

```
const wrapperRef = useRef<HTMLDivElement>(null);
// ...
<div ref={wrapperRef}> ... </div>
// later: wrapperRef.current is the actual <div> DOM node
```

The same file also uses `useRef` for a non-DOM purpose — remembering a value across renders without causing a re-render when it's updated (state via `useState` would re-render on every change; a ref won't):

```
const boundarySnapshotRef = useRef<BoundaryCandidate[]>([]);
// mutated directly during a drag: boundarySnapshotRef.current = [...]
// reading/writing .current never schedules a re-render, unlike a useState setter
```

ANGULAR	REACT
`@ViewChild('wrapper') wrapper!: ElementRef;`, then `this.wrapper.nativeElement`	`const wrapperRef = useRef(null)`, attached via `ref={'{'}wrapperRef{'}'}`, read via `wrapperRef.current`
A plain class field you mutate outside change detection's notice (rare, usually intentional)	`useRef` for any value that should persist across renders but never itself cause one

> **REFS ARE AN ESCAPE HATCH, NOT THE DEFAULT**
>
> If a value should show up in the UI, it belongs in `useState` (or the Zustand store), not a ref — mutating `.current` does not re-render anything. Reach for `useRef` only for DOM access or genuinely "React doesn't need to know this changed" bookkeeping, exactly as `boundarySnapshotRef` is used here for drag-frame math that has no business re-rendering the component 60 times a second.

14 Code-splitting with `React.lazy`

Angular's router lazily loads a feature module's bundle only when its route is visited. React has the same idea at the component level: `lazy()` wraps a dynamic `import()`, and `<Suspense>` declares a fallback to show while that chunk downloads.

◆ src/dashboard/DashboardShell.tsx

```
// Lazy: three.js + drei are a substantial download that only the 3D tab needs — nobody who
// stays on the 2D diagram should pay for it.
const ThreeDView = lazy(() => import('../components/threeDView'));

// ...later, wherever it's rendered:
<Suspense fallback={<TableSkeleton />}>
  <ThreeDView ... />
</Suspense>
```

ANGULAR	REACT
`loadChildren: () => import('./three-d/three-d.module').then(m => m.ThreeDModule)`	`lazy(() => import('../components/threeDView'))` — same dynamic `import()` mechanism under the hood, Vite/webpack split a separate chunk either way
Router shows a loading indicator via a route resolver or a global loading state	`<Suspense fallback={'{'}...{'}'}>` declares the loading UI right where the lazy component is used

This is triggered by a real product concern, not just a demo of the feature: the Three.js/ `@react-three/fiber` 3D view is a heavy dependency (see Chapter 15), so it's excluded from the initial bundle entirely.

15 Wrapping third-party libraries as components

Angular typically wraps a non-Angular library behind a directive or a service. In React, the common pattern is that the library itself ships as React components and hooks — you configure it through props, the same as any of your own components. This app leans on three such libraries:

ReactFlow — the diagram canvas

`src/components/Canvas.tsx` renders `<ReactFlow>` and configures it entirely through props: `nodeTypes` / `edgeTypes` maps your own components (`AwsServiceNode` , `FlowEdge`) to string keys used in the data, and hooks like `useReactFlow()` , `useViewport()` , `useNodesInitialized()` expose the library's internal state the same way `useDiagramStore` exposes this app's own state.

```
const nodeTypes: NodeTypes = { awsService: AwsServiceNode, groupBox: GroupBoxNode, labelNode:
LabelNode };
const edgeTypes: EdgeTypes = { flowEdge: FlowEdge };
// each AwsNode's `type: 'awsService'` field is how ReactFlow picks which component renders it
```

Monaco Editor — Terraform/code preview

`@monaco-editor/react` wraps VS Code's actual editor as one component: `<Editor language="hcl" value={'{'}code{'}'} onChange={'{'}...{'}'} />` in `DashboardShell.tsx` . No separate Angular-style wrapper module to configure — it's imported and rendered exactly like any local component.

react-three-fiber — the 3D view

`src/components/threeDView/` uses `@react-three/fiber` , which is the most conceptually unusual one: it lets you describe a Three.js scene graph *as JSX* (`<mesh>` , `<boxGeometry>`) instead of imperative Three.js calls, reconciled by React the same way DOM elements are. There's genuinely no close Angular analogue to this — Angular has no equivalent renderer-swapping mechanism.

> **THE PATTERN TO TAKE AWAY**
> In React, "integrating a third-party UI library" usually just means "importing components it exports" rather than wrapping it yourself. Configuration happens through props exactly like your own components — there's no separate module/provider registration step most of the time.

16 TypeScript patterns you'll see everywhere

TypeScript itself is identical to what you already know from Angular. What differs is which patterns this codebase leans on — mainly string-literal union types instead of enums, used pervasively for anything with a fixed set of variants.

◆ src/types.ts

```ts
export type ServiceCategory =
  | 'Compute' | 'Networking' | 'Storage' | 'DB'
  | 'Messaging' | 'Security' | 'DevOps' | 'Analytics';

export type ToolMode = 'select' | 'connect' | 'group' | 'label';

export type AwsField = {
  key: string;
  label: string;
  type: 'text' | 'select' | 'number' | 'json' | 'iam-role' | 'file-path';
  options?: string[];
  required?: boolean;
};
```

ANGULAR CODEBASES OFTEN PREFER	THIS CODEBASE PREFERS
`enum ToolMode {'{'} Select, Connect, Group, Label {'}'}`	`type ToolMode = 'select' | 'connect' | 'group' | 'label'` — no runtime object at all, purely a compile-time check, and the values are the exact strings used in JSX/data (no `ToolMode.Select` indirection)
Interfaces for most shapes	Mostly `type` aliases (interfaces do appear, but `type` is the default here) — functionally near-identical for object shapes; this is a style choice, not a technical requirement of React

Props types are usually written inline for small components (as in `Sidebar.tsx`, Chapter 4) and extracted to a named type for larger ones. Component return types are rarely annotated explicitly — TypeScript infers "this returns JSX" from the JSX itself, the same way you'd rarely annotate an Angular getter's return type when it's obvious from its body.

17 Styling: no view encapsulation

Angular scopes a component's CSS to that component by default (`ViewEncapsulation.Emulated`). React has no such mechanism built in — CSS here is plain, global, cascading stylesheets, imported once in `main.tsx` :

```
import './styles.css';
// After styles.css on purpose: builder rules win where the two overlap.
import './styles/builder.css';
```

Scoping is achieved entirely by naming discipline — BEM-ish class names like `service-item__lock` , `sidebar--collapsed` — not by a compiler feature. Theming (dark/light/pearl) is done by toggling a class on a wrapper `<div>` , read back out of `localStorage` :

```
const themeClassName = theme === 'pearl'
  ? 'app-theme app-theme-light app-theme-pearl'
  : `app-theme app-theme-${'$'}{theme} ${'$'}{theme === 'dark' ? 'dark' : ''}`;

return <div className={themeClassName}>{page}</div>;
```

> **NOT EVERY REACT APP DOES IT THIS WAY**
>
> CSS Modules, styled-components, and Tailwind are all common alternatives that *do* give you scoping (each solving it differently — generated unique class names, CSS-in-JS, or utility classes). This codebase uses none of them — just hand-written global CSS files and consistent naming. Worth knowing so you don't assume this is "the React way" rather than "this project's way."

18 A guided reading path through this codebase

Open these files in this order. Each step builds on a concept from an earlier chapter — by the end you'll have traced one full user action (dragging an AWS service onto the canvas) through every layer.

1. `src/main.tsx` — the mount point. Confirm you can see the provider nesting (Ch. 2).
2. `src/App.tsx` — the routing `if` chain and the auth-guard `useEffect` (Ch. 6, Ch. 9). Notice how little code is needed without a router library.
3. `src/dashboard/DashboardShell.tsx` — skim the imports first; it composes almost every other dashboard file. Find the `lazy()` call for `ThreeDView` (Ch. 14).
4. `src/components/Sidebar.tsx` — the smallest complete example of props, `useState`, `useMemo`, and a Zustand selector all in one short file (Ch. 4, 5, 7, 8).
5. `src/store/diagramStore.ts` — read the `DiagramStore` type first (state + action signatures), then pick one action like `addServiceNode` and read its implementation.
6. `src/components/Canvas.tsx` — where the sidebar's drag-and-drop lands. Find the `onDrop` handler and trace it into the store action from step 5.
7. `src/components/canvasGraphContext.ts` — small enough to read end to end in five minutes; ties directly back to what you just saw `lodBand` do in Canvas (Ch. 12).
8. `src/auth/AuthPage.tsx` — the most complete controlled-form example in the app (Ch. 5, 11).

> **HOW TO READ A COMPONENT YOU'VE NEVER SEEN**
>
> Every time: (1) read the props destructured at the top of the function — that's the component's public API, equivalent to skimming an Angular class's `@Input()` / `@Output()` list first; (2) scan the `useState` / `useRef` calls to see what it remembers; (3) scan the `useEffect` calls to see what side effects it triggers and when; (4) only then read the returned JSX. This mirrors how you'd already read an unfamiliar Angular component — inputs/outputs, then fields, then lifecycle hooks, then template — just with all four in one file instead of two.

19 Exercises inside this exact repo

Reading builds recognition; changing code builds fluency. These are scoped to be small, safe, and done entirely within files already covered above — run `npm run dev` and try each one.

1. Add a fourth theme toggle state

easy

`src/theme.ts` defines `themeModes` and `getNextTheme`. Add a new mode (e.g. `'contrast'`) to the type and the cycling function, then add matching CSS. This exercises plain TS unions (Ch. 16) with zero component-lifecycle risk.

2. Add a "clear search" button to the sidebar

easy

In `Sidebar.tsx`, add a button next to the search input that calls `setQuery('')` when `query` is non-empty. Practices `useState` + conditional rendering (Ch. 5, 10) in the smallest possible surface area.

3. Memoize a derived count

medium

Find a place in `Sidebar.tsx` or `DashboardShell.tsx` where a value is recomputed inline on every render (search for an expression not wrapped in `useMemo`) and wrap it, matching the pattern from `filtered` / `counts` (Ch. 7). Verify with React DevTools' profiler that renders actually drop.

4. Add a new context

medium

Create a fourth context alongside the three in `canvasGraphContext.ts` — for example, a `SelectionCountContext` exposing how many nodes are currently selected — and consume it from one node component. Forces you to write both halves: `createContext` + `Provider` + `useContext` (Ch. 12).

5. Add a new protected route

hard

Following the exact shape of the `/settings` guard in `App.tsx`, add a new path (e.g. `/billing`) that requires auth, shows a "Checking access…" state, and redirects unauthenticated users to `/login?next=...`. This is the fullest exercise of Ch. 6 and Ch. 9 together — you'll hit the stale-response race condition firsthand if you forget the `isCurrent` flag.

6. Add a new Zustand action with undo support

hard

In `diagramStore.ts`, add a small new action (e.g. `renameSelectedNode(name: string)`) that goes through the same `pushHistory` path as the existing mutation actions, so `undo()` / `redo()` correctly reverts it. Read `updateNodeData` first as your template.

20 Cheat sheet

One page. Everything above, compressed.

ANGULAR	REACT
Component class + template file	One function returning JSX
`@Input()`	Function parameter (destructured from `props`)
`@Output()` + `EventEmitter`	A function prop, called directly
Class field + change detection	`useState` — mutate only via its setter
`ngOnInit`	`useEffect(fn, [])`
`ngOnChanges`	`useEffect(fn, [depsThatMatter])`
`ngOnDestroy`	The function `useEffect`'s callback returns
Pure pipe / computed signal	`useMemo(fn, deps)`
Injectable service (+ optional RxJS subject / NgRx)	A `zustand` (or similar) store, imported directly
`RouterModule` / `routerLink`	Either `react-router` (common elsewhere) or, as here, manual `window.location.pathname` checks
`*ngFor` (+ `trackBy`)	`array.map()` (+ mandatory `key` prop)
`*ngIf`	`condition && <X/>`, a ternary, or an early `return`
`[(ngModel)]`	`value={'{'}x{'}'} onChange={'{'}(e)=>setX(e.target.value){'}'}`
DI token / provider	`createContext` + `<Provider>` + `useContext`
`@ViewChild`	`useRef`
Lazy-loaded route module	`React.lazy(() => import(...))` + `<Suspense>`
`ViewEncapsulation.Emulated`	None built in — global CSS + naming discipline (this repo), or CSS Modules/styled-components/Tailwind (common elsewhere)
Zone.js auto-detects changes	Nothing is automatic — a re-render only happens because a `useState`/store setter was called